

Universidad Rafael Landívar  
Facultad de Ingeniería  
Manejo de Archivos

# Proyecto 1

## Documentación externa

## FUNCIONALIDAD

### **main.py**

Muestra el menú, válida la opción, llama al módulo correspondiente y captura excepciones para que el programa no se cierren, en resumen, es el punto de conexión entre las clases y el usuario.

Menú:

1. Registrar información
2. Mostrar todos los registros
3. Buscar por ID (secuencial / índice / hash)
4. Buscar por atributo (secuencial / invertido / multillave)
5. Actualizar registro
6. Eliminar registro
7. Consultar índices
8. Generar / reconstruir índices
9. Consultar tabla hash
10. Organización tipo pila
11. Verificar integridad de archivos
12. Generar respaldo
13. Restaurar respaldo
14. Consultar metadatos
15. Consultar logs
16. Exportar a XML
17. Ver configuración (JSON)
18. Salir

## **archivos.py**

Archivo: data/registros.csv

Estructura: lista de diccionarios.

### **asegurar\_estructura()**

Parámetros: ninguno

Retorno: ninguno

Qué hace: Crea carpetas y el CSV con su encabezado si no existen.

Por qué: evita errores de ejecución en la primera corrida.

### **leer\_registros()**

Parámetros: ninguno

Retorno: list[dict]

Qué hace: Abre el CSV con csv.DictReader, recorre todas las filas y lo cierra con with.

Por qué: para leer los registros desde el archivo principal.

### **leer\_registro\_en\_posicion(pos)**

Parámetros: número de fila

Retorno: dict / None

Qué hace: Recorre hasta la posición indicada y devuelve ese registro.

Por qué: para buscar por índice.

### **escribir\_registros(registros)**

Parámetros: lista de dicts

Retorno: ninguno

Qué hace: Reescribe el CSV completo en un archivo temporal y luego lo reemplaza con os.replace.

Por qué: para no dejar un CSV a medias si algo falla.

### **agregar\_registro(registro)**

Parámetros: dict

Retorno: bool

Qué hace: Añade una fila al final del CSV usando el modo de apertura 'a'.

Por qué: para agregar sin reescribir todo el archivo.

### **validar\_registro(registro, existentes)**

Parámetros: dict, list

Retorno: (bool, mensaje)

Qué hace: Verifica campos obligatorios, estado válido, fecha AAAA-MM-DD e ID no duplicado.

Por qué: para evitar IDs duplicados y datos inconsistentes.

### **exportar\_xml(ruta=None)**

Parámetros: ruta opcional

Retorno: str (ruta generada)

Qué hace: Convierte el CSV a XML con xml.etree.ElementTree.

Por qué: para transferir los datos a otro formato de intercambio.

## **operaciones.py**

Relación: es el único lugar donde una operación que cambia datos actualiza todas las estructuras auxiliares.

### **registrar\_equipo(datos)**

Parámetros: dict

Retorno: (bool, str)

Qué hace: Valida, agrega al CSV, llama a sincronizar\_estructuras(), registra log CREATE y hace push en la pila.

Por qué: para mantener consistencia entre CSV, índices, hash, hashes de integridad y pila.

### **modificar\_equipo(id, cambios)**

Parámetros: ID, dict de campos

Retorno: (bool, str)

Qué hace: valida el id, reescribe el CSV, sincroniza, registra log UPDATE.

Por qué: para que cualquier cambio se refleje en todas las estructuras.

### **eliminar\_equipo(id)**

Parámetros: ID

Retorno: (bool, str)

Qué hace: Verifica que exista, reescribe el CSV sin ese registro, sincroniza, registra log delete y hace push.

Por qué: para no dejar índices apuntando a filas inexistentes.

### **sincronizar\_estructuras()**

Parámetros: ninguno

Retorno: ninguno

Qué hace: Reconstruye índices, reconstruye la tabla hash y actualiza los hashes de SHA-256.

Por qué: para sincronizar y para evitar inconsistencias entre el archivo principal y las estructuras auxiliares.

## **utilidades.py**

Archivos: config/configuracion.json

Relación: lo importan todos los demás módulos.

### **cargar\_json(ruta)**

Parámetros: ruta del JSON

Retorno: dict / list / None

Qué hace: Abre, lee y cierra un archivo JSON. Devuelve None si no existe o está dañado.

Por qué: para no cerrar el programa si un archivo auxiliar falla.

### **guardar\_json(ruta, datos)**

Parámetros: ruta, objeto

Retorno: bool

Qué hace: Escribe el JSON ordenado y con sangría.

Por qué: para que sea legible y los hashes SHA-256 sean estables.

### **cargar\_configuracion()**

Parámetros: ninguno

Retorno: dict

Qué hace: Lee configuracion.json y devuelve los parámetros del sistema.

Por qué: para tener un punto único de acceso a la configuración.

### **ahora()**

Parámetros: ninguno

Retorno: str ("AAAA-MM-DD HH:MM:SS")

Qué hace: Obtiene la fecha y hora del sistema.

Por qué: para los logs y nombres de respaldos.

### **marca\_respaldo()**

Parámetros: ninguno

Retorno: str ("AAAAMMDD\_HHMMSS")

Qué hace: Genera la parte automática del nombre de un respaldo.

Por qué: para no escribir manualmente el nombre de cada respaldo.

### **pedir\_texto(msg, obligatorio)**

Parámetros: mensaje, bool

Retorno: str

Qué hace: Pide texto al usuario y repite si se deja vacío.

Por qué: para evitar entradas inválidas.

### **pedir\_opcion(msg, validas)**

Parámetros: mensaje, lista

Retorno: str

Qué hace: Pide una opción y repite hasta recibir una válida.

Por qué: para controlar opciones incorrectas del menú.

## logs.py

Archivo: logs/sistema.log (modo agregar 'a': nunca se sobrescribe).

### **registrar\_log(tipo, descripcion)**

Parámetros: tipo (operacion, descripción), texto

Retorno: nada

Qué hace: Añade una línea con formato fecha hora | TIPO | descripción.

Por qué: para registrar automáticamente cada operación relevante.

### **leer\_logs()**

Parámetros: cantidad de líneas

Retorno: list[str]

Qué hace: Devuelve las últimas líneas del log.

Por qué: para mostrarlas en el menú.

## busquedas.py

Todas devuelven un diccionario con: valor\_buscado, encontrado, registros, recorridos y metodo.

### **buscar\_secuencial\_id(id)**

Parámetros: ID

Retorno: dict

Qué hace: Lee el CSV fila por fila hasta encontrar el ID.

Por qué: para demostrar el acceso secuencial y contar registros recorridos.

### **buscar\_por\_indice\_id(id)**

Parámetros: ID

Retorno: dict

Qué hace: Consulta indice\_id.json, obtiene posición y lee esa fila.

Por qué: para localizar información más rápido que la búsqueda secuencial.

### **buscar\_por\_hash(id)**

Parámetros: ID

Retorno: dict

Qué hace: Calcula posición hash y recorre la cadena de esa posición.

Por qué: para demostrar el uso de hashing como mecanismo de localización.

### **buscar\_atributo\_secuencial(campo, valor)**

Parámetros: campo, valor

Retorno: dict

Qué hace: Recorre todo el CSV filtrando por el campo indicado.

Por qué: para atributos que no tienen índice.

**buscar\_invertido(campo, valor)**

Parámetros: Categoría/Estado, valor

Retorno: dict

Qué hace: Consulta el índice invertido, obtiene lista de IDs y recupera esos registros.

Por qué: para consultar por atributo usando índice invertido.

**buscar\_multillave(criterios)**

Parámetros: dict

Retorno: dict

Qué hace: Obtiene el conjunto de IDs de cada índice y los intersecta.

Por qué: para demostrar organización multillave.

**indices.py**

Archivos: indices/indice\_id.json, indice\_categoria.json, indice\_estado.json.

**construir\_indice\_id(registros)**

Parámetros: lista

Retorno: dict {id: posicion}

Qué hace: Recorre los registros y anota la posición de cada ID.

Por qué: para tener el índice principal ID a posición.

**construir\_indice\_invertido(registros, campo)**

Parámetros: lista, campo

Retorno: dict {valor: [ids]}

Qué hace: Agrupa los IDs según el valor del campo.

Por qué: para consultas por categoría o estado.

**generar\_indices()**

Parámetros: ninguno

Retorno: dict resumen

Qué hace: Construye y guarda los tres índices. Registra log INDEX.

Por qué: para dejar las estructuras auxiliares listas.

**reconstruir\_indices()**

Parámetros: ninguno

Retorno: bool

Qué hace: Lee todo el CSV, regenera y guarda todo. Registra log REINDEX.

Por qué: es la salvación si un índice se borra o se daña.

**cargar\_indice(nombre)**

Parámetros: nombre

Retorno: dict / None

Qué hace: Lee un índice; devuelve None si no existe.

Por qué: para validar existencia antes de usarlo.

### **indices\_desactualizados()**

Parámetros: ninguno

Retorno: list[str]

Qué hace: Compara índices contra el CSV para verificar valores faltantes o incongruencias

Por qué: para detectar inconsistencias y reconstruir.

## **hashing.py**

Archivo: indices/tabla\_hash.json

Estructura: diccionario {posicion: [entradas]}.

### **funcion\_hash(clave, tamano)**

Parámetros: ID (texto), tamaño de tabla

Retorno: int

Qué hace: Suma los códigos ASCII de cada carácter y toma el residuo entre el tamaño.

Por qué: para calcular la posición de una clave de forma explicable y estable.

### **construir\_tabla\_hash(registros, tamano)**

Parámetros: lista, tamaño

Retorno: dict

Qué hace: Inserta cada ID en su posición; si ya hay algo, lo añade a la cadena.

Por qué: para manejar colisiones mediante encadenamiento.

### **buscar\_en\_tabla(id)**

Parámetros: ID

Retorno: dict

Qué hace: Calcula posición y recorre solo esa cadena.

Por qué: para localizar sin recorrer todo el CSV.

### **detectar\_colisiones(tabla)**

Parámetros: tabla

Retorno: list

Qué hace: Lista las posiciones con más de una clave.

Por qué: para mostrarlas en la defensa.

## **integridad.py**

Archivos: los de archivosProtegidos + indices/hashes.json.

### **calcular\_sha256(ruta)**

Parámetros: ruta

Retorno: str / None

Qué hace: Lee el archivo en bloques de 4096 bytes mediante hashlib.sha256().

Por qué: para generar la huella digital sin cargar todo en memoria.

### **actualizar\_hashes()**

Parámetros: ninguno

Retorno: dict

Qué hace: Recalcula y guarda las huellas de los archivos protegidos.

Por qué: solo la llama el sistema tras una operación legítima.

### **verificar\_integridad()**

Parámetros: ninguno

Retorno: list[dict]

Qué hace: Verifica la integridad del archivo comparando las huellas digitales de esta antes y después de un posible cambio

Por qué: para detectar ediciones manuales externas.

## **pila.py**

Archivo: data/historial\_pila.json

Estructura: lista de Python donde el final es el TOP.

### **push(elemento)**

Parámetros: elemento

Retorno: None

Qué hace: Agrega al TOP y guarda en el archivo.

Por qué: para registrar cada operación que cambia datos.

### **pop()**

Parámetros: ninguno

Retorno: elemento / None

Qué hace: Quita y devuelve el TOP.

Por qué: para demostrar el comportamiento LIFO.

### **peek()**

Parámetros: ninguno

Retorno: elemento / None

Qué hace: Muestra el TOP sin quitarlo.

Por qué: para consultar la operación más reciente.

**tamano() /**

Parámetros: ninguno

Retorno: bool / int

Qué hace: Indica el estado de la pila.

Por qué: para controlar la estructura antes de operar.

**listar()**

Parámetros: ninguno

Retorno: list

Qué hace: Devuelve los elementos del TOP hacia abajo.

Por qué: para mostrarlos en el menú.

**respaldos.py**

Archivos: backups/backup\_AAAAMMDD\_HHMMSS/

**crear\_respaldo()**

Parámetros: ninguno

Retorno: str (ruta)

Qué hace: Genera el nombre automático con la hora del sistema; copia con shutil.copytree/copy2. Registra log BACKUP.

Por qué: para conservar una copia fechada de los archivos importantes.

**listar\_respaldos()**

Parámetros: ninguno

Retorno: list[str]

Qué hace: Devuelve las carpetas existentes ordenadas de la más antigua a la más reciente.

Por qué: para seleccionar cuál restaurar.

**restaurar\_respaldo(nombre)**

Parámetros: nombre

Retorno: bool

Qué hace: Verifica la existencia del respaldo y verifica su integridad, si es íntegro se carga, actualizando los índices y tabla hash. Registra log RESTORE.

Por qué: para recuperar información real sin dejar el sistema a medias.

## **metadatos.py**

### **obtener\_metadatos(ruta)**

Parámetros: ruta

Retorno: dict

Qué hace: Con pathlib/os.stat obtiene nombre, ruta absoluta, tamaño, extensión, fecha de creación, fecha de modificación y permisos.

Por qué: para consultar propiedades de archivos reales del sistema.

### **listar\_metadatos\_sistema()**

Parámetros: ninguno

Retorno: list[dict]

Qué hace: Devuelve metadatos de los archivos reales del sistema (CSV, JSON, índices y log).

Por qué: para mostrar un resumen general.

## **Organización secuencial y acceso secuencial**

El CSV guarda los registros uno tras otro y la búsqueda secuencial los recorre fila por fila desde el inicio hasta encontrar el ID o llegar al final, contando cuántos registros revisó.

## **Organización secuencial indexada**

El índice principal se construye leyendo el CSV una sola vez y guardando en indice\_id.json la relación ID con su posición, sin copiar los datos. Al consultar un ID se obtiene su posición y se lee solo esa fila, así que el CSV sigue siendo la fuente de verdad. Como las posiciones cambian al agregar, modificar o eliminar, los índices se reconstruyen siempre.

## **Hashing y colisiones**

Se usa una función hash propia que suma los códigos ASCII del ID y aplica módulo 10 para obtener la posición en tabla\_hash.json. Cuando dos IDs caen en la misma posición se produce una colisión, que se resuelve con encadenamiento: cada posición guarda una lista de entradas y la búsqueda las compara una por una.

## **Organización multillave**

Un mismo registro se puede localizar por distintas claves gracias a varios índices: ID con posición, categoría con lista de IDs, estado con lista de IDs y hash con cadena de entradas. La búsqueda multillave consulta los índices necesarios y cruza los resultados.

**Pila**

La pila se guarda en historial\_pila.json y registra cada operación que cambia datos (crear, modificar, eliminar). Cada nueva operación se agrega al final con push, y el último elemento es el TOP. Con pop se saca el último y con peek solo se mira.

**Integridad SHA-256**

Se calcula la huella SHA-256 de cada archivo protegido y se guarda en hashes.json. Al verificar, se recalcula y se compara: si coincide es OK, si no, es ALTERADO. Las operaciones normales del sistema actualizan las huellas, pero una edición manual externa no, y por eso se detecta la modificación.

## ETAPAS

### Etaapa 1 — Preparación del entorno

1. Qué se hace: verificar Python, Git y editor, y configurar identidad de Git.
2. Comando usado: `python --version`, `git --version`, `git config --list`
3. Qué hace: muestra las versiones instaladas y configuración de visual durante la creación del programa.
4. Por qué se usa: el proyecto requiere Python + Git + VS Code, los commits requieren una entidad a la cual ligarse.
5. Resultado: versiones impresas (el proyecto fue ejecutado con Python 3.14.6 según los .pyc) e identidad configurada.
6. Demuestra: el entorno cumple los requisitos y es reproducible.

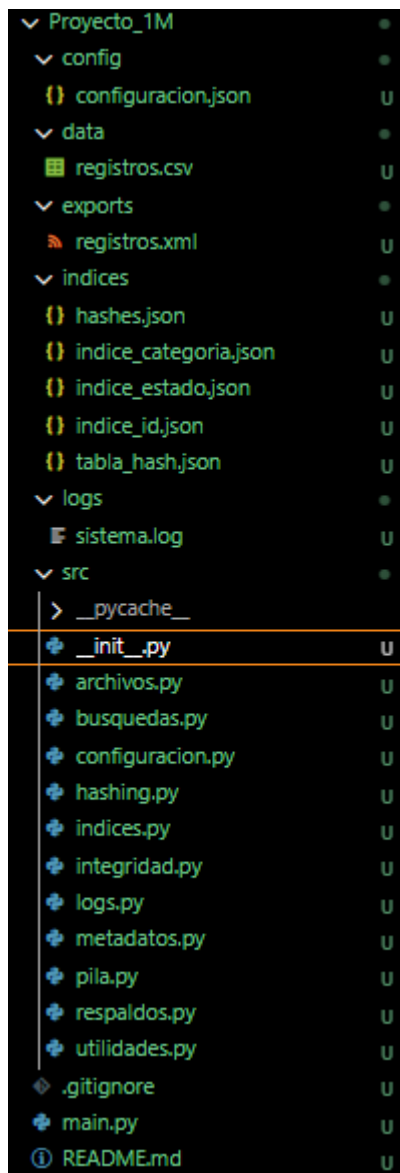
```
vicme@LAPTOP-362BMOJA MINGW64 ~/OneDrive/Escritorio/ManejoEImplementacion/manejo_implementacion_archivos_url_s202602 (
1236925_MELGAR_CASTILLO_MAX_ANDRES)
● $ python --version
Python 3.14.6

vicme@LAPTOP-362BMOJA MINGW64 ~/OneDrive/Escritorio/ManejoEImplementacion/manejo_implementacion_archivos_url_s202602 (
1236925_MELGAR_CASTILLO_MAX_ANDRES)
● $ git --version
git version 2.45.1.windows.1
```

```
vicme@LAPTOP-362BMOJA MINGW64 ~/OneDrive/Escritorio/ManejoEImplementacion/manejo_implementacion_archivos_url_s202602 (
1236925_MELGAR_CASTILLO_MAX_ANDRES)
● $ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=Max
user.email=azmelgarcastillo@gmail.com
core.repositoryformatversion=0
:...skipping...
```

## Etapa 2 — Creación del proyecto

1. **Qué se hace:** crear la carpeta principal y la estructura de directorios/archivos.
2. **Comando usado:** creación manual (no se usan comandos)
3. **Qué hace:** crea las carpetas y las futuras clases a usar para el funcionamiento
4. **Por qué se usa:** es buena práctica al ordenar las diferentes partes en carpetas
5. **Resultado:** estructura de carpeta de proyecto ordenada
6. **Demuestra:** organización modular



### **Etapas 3 — Inicialización de Git**

1. **Qué se hace:** crear el repositorio local y el primer commit.
2. **Comando usado:** git init · git status · git add . git commit -m "Inicializacion y estructura proyecto"
3. **Qué hace:** init crea, .git status lista archivos sin seguimiento, add los prepara y commit guarda la primera versión.
4. **Por qué se usa:** Requisito de proyecto, mantiene un historial de git
5. **Resultado:** repositorio inicializado en rama main con primer commit.
6. **Demuestra:** control de versiones desde el principio.

```

vicme@LAPTOP-362BMOJA MINGW64 ~/OneDrive/Escritorio/ManejoImplementacion/manejo_implementacion_archivos_url_s202602
1236925_MELGAR_CASTILLO_MAX_ANDRES)
$ git status
deleted:    Proyecto_1/indices/indice_estado.json
deleted:    Proyecto_1/indices/indice_id.json
deleted:    Proyecto_1/indices/tabla_hash.json
deleted:    Proyecto_1/logs/sistema.log
deleted:    Proyecto_1/main.py
deleted:    Proyecto_1/src/__init__.py
deleted:    Proyecto_1/src/_pycache_/__init__.cpython-314.pyc
deleted:    Proyecto_1/src/_pycache_/archivos.cpython-314.pyc
deleted:    Proyecto_1/src/_pycache_/busquedas.cpython-314.pyc
deleted:    Proyecto_1/src/_pycache_/indices.cpython-314.pyc
deleted:    Proyecto_1/src/archivos.py
deleted:    Proyecto_1/src/busquedas.py
deleted:    Proyecto_1/src/hashing.py
deleted:    Proyecto_1/src/indices.py
deleted:    Proyecto_1/src/integridad.py
deleted:    Proyecto_1/src/logs.py
deleted:    Proyecto_1/src/metadatos.py
deleted:    Proyecto_1/src/pila.py
deleted:    Proyecto_1/src/respaldos.py
deleted:    Proyecto_1/src/utilidades.py

```

```

1236925_MELGAR_CASTILLO_MAX_ANDRES)
$ git add .

vicme@LAPTOP-362BMOJA MINGW64 ~/OneDrive/Escritorio/ManejoImplementacion/manejo_implementacion_archivos_url_s202602
1236925_MELGAR_CASTILLO_MAX_ANDRES)
$ git commit -m "Iniciación de estructura proyecto"
[1236925_MELGAR_CASTILLO_MAX_ANDRES 14f247c] Inicialización de estructura proyecto
47 files changed, 2627 insertions(+), 2232 deletions(-)
delete mode 100644 Proyecto_1/data/registros.csv
delete mode 100644 Proyecto_1/indices/indices.json
delete mode 100644 Proyecto_1/indices/indice_categoria.json
delete mode 100644 Proyecto_1/indices/indice_estado.json
delete mode 100644 Proyecto_1/indices/indice_id.json
delete mode 100644 Proyecto_1/indices/tabla_hash.json
delete mode 100644 Proyecto_1/logs/sistema.log
delete mode 100644 Proyecto_1/main.py
delete mode 100644 Proyecto_1/src/_pycache_/__init__.cpython-314.pyc
delete mode 100644 Proyecto_1/src/_pycache_/archivos.cpython-314.pyc
delete mode 100644 Proyecto_1/src/_pycache_/busquedas.cpython-314.pyc
delete mode 100644 Proyecto_1/src/_pycache_/indices.cpython-314.pyc
delete mode 100644 Proyecto_1/src/archivos.py
delete mode 100644 Proyecto_1/src/busquedas.py
delete mode 100644 Proyecto_1/src/hashing.py
delete mode 100644 Proyecto_1/src/indices.py

```

```

$ git commit -m "Iniciación de estructura proyecto"
rename {Proyecto_1 => proyecto_1_1236925_max_andres_melgar_castillo}/config/configuracion.json (100%)
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/data/registros.csv
rename {Proyecto_1 => proyecto_1_1236925_max_andres_melgar_castillo}/exports/registros.xml (100%)
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/indices/indices.json
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/indices/indice_categoria.json
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/indices/indice_estado.json
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/indices/indice_id.json
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/indices/tabla_hash.json
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/logs/sistema.log
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/main.py
rename {Proyecto_1 => proyecto_1_1236925_max_andres_melgar_castillo}/src/__init__.py (100%)
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/src/archivos.py
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/src/busquedas.py
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/src/configuracion.py
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/src/hashing.py
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/src/indices.py
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/src/integridad.py
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/src/logs.py
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/src/metadatos.py
rename {Proyecto_1 => proyecto_1_1236925_max_andres_melgar_castillo}/src/pila.py (100%)
create mode 100644 proyecto_1_1236925_max_andres_melgar_castillo/src/respaldos.py

```

## Etapa 4 – Archivo principal CSV

1. **Qué se hace:** crear/leer/escribir data/registros.csv.
2. **Comando usado:** creacion opciones 1 y 2 del menu del main
3. **Qué hace:** crear\_archivo\_si\_no\_existe() crea el CSV con encabezado id,nombre,categoria,descripcion,fecha,estado, leer\_registros() lo recorre con csv.DictReader y agregar\_registro() añade al final.
4. **Por qué se usa:** ingreso de datos y muestra de estos
5. **Resultado:** ingreso de registros a csv y impresion de estos
6. **Demuestra:** capacidad de retener datos mediante el csv.

```
=====
Seleccione una opción: 1

=== REGISTRAR INFORMACIÓN ===
(Escriba 'cancelar' en cualquier campo para volver)
ID (numérico): 01
Nombre: Max
Categoría: algo
Descripción: si
Fecha (AAAA-MM-DD): 2025-05-12
Estado: activo

Registro almacenado correctamente.
Índices actualizados correctamente.
Tabla hash actualizada correctamente.
Hashes de integridad actualizados.
```

```
data >  registros.csv
1 id,nombre,categoria,descripcion,fecha,estado
2 01,Max,algo,si,2025-05-12,activo
3 
```

## Etapa 5 — Operaciones básicas

1. **Qué se hace:** crear, consultar, modificar y eliminar un registro.
2. **Comando usado:** creación opción 1,3,5,6
3. **Qué hace:** opción 1 (crear), 3 (consultar), 5 (actualizar), 6 (eliminar).
4. **Por qué se usa:** para mantener las estructuras actualizadas y evitar incongruencias entre huellas.
5. **Resultado:** adición de datos a tabla hash, modificación de índices,
6. **Demuestra:** habilidades para modificar o trabajar con los datos guardados en las estructuras

```
Seleccione una opción: 2

=== REGISTROS ALMACENADOS ===

-----
ID:      01
Nombre:  Max
Categoría: algo
Descripción: si
Fecha:   2025-05-12
Estado:  activo
-----

Total de registros: 1
```

```
Seleccione una opción: 5

=== ACTUALIZAR REGISTRO ===
Ingrese el ID: 01

Información actual:

-----
ID:      01
Nombre:  Max
Categoría: algo
Descripción: si
Fecha:   2025-05-12
Estado:  activo
-----

Ingrese la nueva información (Enter = conservar el valor actual, 'cancelar' = volver):
Nombre [Max]: Melgar
Categoría [algo]: quiza
Descripción [si]: no
Fecha (AAAA-MM-DD) [2025-05-12]: 2077-11-15
Estado [activo]: intermitente

Registro actualizado correctamente.
Índices actualizados correctamente.
Tabla hash actualizada correctamente.
Hashes de integridad actualizados.
```

```

Seleccione una opción: 2

=== REGISTROS ALMACENADOS ===

-----
ID:      01
Nombre:  Melgar
Categoría:  quiza
Descripción:  no
Fecha:    2077-11-15
Estado:   intermitente
-----

Total de registros: 1

```

## Etapa 6 — JSON

1. **Qué se hace:** usar config/configuracion.json como configuración leída por el sistema.
2. **Comando usado:** python main.py (se lee al iniciar) · mostrar el archivo.
3. **Qué hace:** cargar la configuración con los parámetros para los archivos de exportación, tamaño tabla ash, título del menú etc.
4. **Por qué se usa:** evita incoherencias con rutas
5. **Resultado:** cambio de tamaño tabla hash o nombre del menú
6. **Demuestra:** que el JSON es parte funcional del sistema

```

{
  "nombre_sistema": "Sistema Integral de Gestión y Recuperación de Información",
  "version": "1.0",
  "archivo_principal": "data/registros.csv",
  "formato_principal": "CSV",
  "archivo_exportacion": "exports/registros.xml",
  "carpeta_indices": "indices",
  "carpeta_logs": "logs",
  "carpeta_respalDOS": "backups",
  "algoritmo_integridad": "SHA-256",
  "tamano_tabla_hash": 10
}

```

```

import json
import os

RUTA_CONFIG = "config/configuracion.json"

VALORES_POR_DEFECTO = {
  "nombre_sistema": "Sistema Integral de Gestión y Recuperación de Información",
  "version": "1.0",
  "archivo_principal": "data/registros.csv",
  "formato_principal": "CSV",
  "archivo_exportacion": "exports/registros.xml",
  "carpeta_indices": "indices",
  "carpeta_logs": "logs",
  "carpeta_respalDOS": "backups",
  "algoritmo_integridad": "SHA-256",
  "tamano_tabla_hash": 10
}

```

```

def tamaño_tabla():
    """
    Tamaño de la tabla hash (debe ser un entero positivo).
    """

    valor = obtener("tamaño_tabla_hash")

    if (
        isinstance(valor, int)
        and not isinstance(valor, bool)
        and valor > 0
    ):
        return valor

    return VALORES_POR_DEFECTO["tamaño_tabla_hash"]

```

```

def ruta_registros():
    return obtener("archivo_principal")

def ruta_xml():
    return obtener("archivo_exportacion")

def carpeta_indices():
    return obtener("carpeta_indices")

def ruta_indice_id():
    return _unir(carpeta_indices(), "indice_id.json")

def ruta_indice_categoria():
    return _unir(carpeta_indices(), "indice_categoria.json")

def ruta_indice_estado():
    return _unir(carpeta_indices(), "indice_estado.json")

def ruta_tabla_hash():
    return _unir(carpeta_indices(), "tabla_hash.json")

def ruta_hashes():
    return _unir(carpeta_indices(), "hashes.json")

def ruta_log():
    return _unir(obtener("carpeta_logs"), "sistema.log")

def carpeta_respalDOS():
    return obtener("carpeta_respalDOS")

```

```

def cargar_configuracion():
    """
    Lee config/configuracion.json y retorna un diccionario.

    - Si el archivo no existe, lo crea con los valores por defecto.
    - Si está dañado o incompleto, usa los valores por defecto
      para las claves que falten (el sistema no se detiene).
    """

    configuracion = dict(VALORES_POR_DEFECTO)

    if not os.path.exists(RUTA_CONFIG):
        try:
            os.makedirs("config", exist_ok=True)

            with open(
                RUTA_CONFIG, "w", encoding="utf-8"
            ) as archivo:
                json.dump(
                    VALORES_POR_DEFECTO,
                    archivo,
                    indent=4,
                    ensure_ascii=False
                )
        except OSError:
            pass

        return configuracion

    try:
        with open(
            RUTA_CONFIG, "r", encoding="utf-8"
        ) as archivo:
            leido = json.load(archivo)

            if isinstance(leido, dict):
                configuracion.update(leido)

    except (OSError, json.JSONDecodeError):
        pass

    return configuracion

```

## Etapla 7 – XML

1. **Qué se hace:** exportar el CSV a XML.
2. **Comando usado:** menú opción 21 (Exportar información a XML).
3. **Qué hace:** exportar\_a\_xml() crea un xml usando elementTree y los datos del csv, recorriendo los datos, traduciendo los a ET y insertando los al xml.
4. **Por qué se usa:** formato de intercambio solicitado
5. **Resultado:** exports/registros.xml
6. **Demuestra:** conversión CSV a XML

```
Seleccione una opción: 2

=== REGISTROS ALMACENADOS ===

-----
ID:      01
Nombre:  Melgar
Categoría: quiza
Descripción: no
Fecha:   2077-11-15
Estado:  intermitente
-----

Total de registros: 1
```

```
Seleccione una opción: 21

=== EXPORTAR INFORMACIÓN A XML ===

Información exportada correctamente.
Archivo generado: exports/registros.xml
```

```
ta > registros.csv
1 id,nombre,categoria,descripcion,fecha,estado
2 01,Melgar,quiza,no,2077-11-15,intermitente
3
```

```
exports > registros.xml
1 <?xml version='1.0' encoding='utf-8'?>
2 <registros>
3   <registro>
4     <id>01</id>
5     <nombre>Melgar</nombre>
6     <categoria>quiza</categoria>
7     <descripcion>no</descripcion>
8     <fecha>2077-11-15</fecha>
9     <estado>intermitente</estado>
10   </registro>
11 </registros>
```

## Etapla 8 — Búsqueda secuencial

1. **Qué se hace:** buscar un ID recorriendo el archivo.
2. **Comando usado:** menú opción 3 (secuencial) y opción 4 (por atributo).
3. **Qué hace:** `busqueda_secuencial_por_id()` itera `leer_registros()`, cuenta recorridos y se detiene al encontrar el ID, retorna (registro, recorridos).
4. **Por qué se usa:** es el método base para comparar contra el índice.
5. **Resultado:** muestra valor buscado, registro, registros recorridos y resultado
6. **Demuestra:** acceso secuencial

```
Seleccione una opción: 3

=== BÚSQUEDA SECUENCIAL POR ID ===
Ingrese el ID: 01

Valor buscado: ID = 01

-----
ID:          01
Nombre:      Melgar
Categoría:   quiza
Descripción: no
Fecha:       2077-11-15
Estado:      intermitente
-----
Resultado: ENCONTRADO
Registros recorridos: 1
```

```
def busqueda_secuencial_por_id(id_buscado):
    """
    Recorre el archivo principal registro por registro
    hasta encontrar el ID.

    Retorna:
    - registro encontrado (o None)
    - cantidad de registros recorridos
    """

    recorridos = 0

    for registro in leer_registros():
        recorridos += 1

        if registro["id"] == id_buscado:
            return registro, recorridos

    return None, recorridos
```

```
Seleccione una opción: 4

=== BUSCAR POR ATRIBUTO (secuencial) ===
1. Nombre
2. Categoría
3. Estado
4. Fecha
Seleccione el atributo: 3
Ingrese estado: intermitente

Se encontraron 1 registro(s):

-----
ID:      01
Nombre:  Melgar
Categoría:  quiza
Descripción: no
Fecha:    2077-11-15
Estado:   intermitente
-----
```

```
def busqueda_por_atributo(atributo, valor):
    """
    Búsqueda secuencial de todos los registros cuyo
    atributo coincide con el valor (sin distinguir
    mayúsculas de minúsculas).
    """

    if atributo not in CAMPOS:
        raise ValueError(f"Atributo desconocido: {atributo}")

    valor = valor.strip().lower()

    return [
        registro
        for registro in leer_registros()
        if registro[atributo].strip().lower() == valor
    ]
```

## Etapla 9 — Índice principal

1. **Qué se hace:** construir y consultar índices/índice\_id.json.
2. **Comando usado:** menú opción 7 (mostrar), 9 (buscar con índice), 10 (reconstruir).
3. **Qué hace:** generar\_indice\_id() recorre el CSV y guarda (ID → posición). buscar\_por\_indice\_id() toma la posición y lee ese registro.
4. **Por qué se usa:** localizar por ID sin recorrer todo
5. **Resultado:** índice JSON coherente con el CSV
6. **Demuestra:** relación ID → registro y que el índice es una estructura auxiliar reconstruible.

```
def generar_indice_id():  
  
    indice = _construir_indices(Leer_registros())[0]  
    _guardar_json(ruta_indice_id(), indice)  
    return indice
```

```
Proyecto_1M > indices > {} índice_id.json > ...  
1  {  
2    "255": 0,  
3    "1919": 1  
4  }
```

```
def buscar_por_indice_id(id_buscado):  
    """  
    Búsqueda con el índice principal (ID -> posición).  
  
    Retorna (registro, posición) o (None, None).  
    """  
  
    indice = cargar_indice(ruta_indice_id()) or {}  
  
    if id_buscado not in indice:  
        return None, None  
  
    posicion = indice[id_buscado]  
  
    registros = leer_registros()  
  
    if posicion >= len(registros):  
        return None, None  
  
    registro = registros[posicion]  
  
    if registro["id"] != id_buscado:  
        return None, None  
  
    return registro, posicion
```

Seleccione una opción: 7

=== ÍNDICE PRINCIPAL (ID -> posición) ===  
ID: 01 -> Posición: 0

Seleccione una opción: 9

=== BÚSQUEDA UTILIZANDO ÍNDICES ===

1. ID (índice principal)
2. Categoría (índice invertido)
3. Estado (índice invertido)

Seleccione el tipo de búsqueda: 2

Ingrese la categoría: quizá

Se encontraron 1 registro(s):

01 - Melgar

## Etapa 10 — Organización secuencial indexada

1. **Qué se hace:** comparar búsqueda por índice vs. secuencial.
2. **Comando usado:** menú opción 3 y opción 9 con el mismo ID.
3. **Qué hace:** el índice entrega la posición y se accede directamente al registro; la secuencial lee desde el inicio.
4. **Por qué se usa:** el PDF pide explicar construcción, contenido, consulta y qué pasa al agregar/modificar/eliminar.
5. **Resultado:** mismo registro con distinta cantidad de registros revisados.
6. **Demuestra:** CSV + índice = organización secuencial indexada.

```
Seleccione una opción: 1

=== REGISTRAR INFORMACIÓN ===
(Escriba 'cancelar' en cualquier campo para volver)
ID (numérico): 02
Nombre:
  X El nombre no puede estar vacío. (escriba 'cancelar' para volver)
Nombre: Pepe
Categoría: alumno
Descripción: sistemas
Fecha (AAAA-MM-DD): 2005-11-10
Estado: activo

Registro almacenado correctamente.
Índices actualizados correctamente.
Tabla hash actualizada correctamente.
Hashes de integridad actualizados.
```

```
Seleccione una opción: 3

=== BÚSQUEDA SECUENCIAL POR ID ===
Ingrese el ID: 02

Valor buscado: ID = 02

-----
ID:      02
Nombre:  Pepe
Categoría: alumno
Descripción: sistemas
Fecha:   2005-11-10
Estado:  activo
-----

Resultado: ENCONTRADO
Registros recorridos: 2
```

```
Seleccione una opción: 9

=== BÚSQUEDA UTILIZANDO ÍNDICES ===
1. ID (índice principal)
2. Categoría (índice invertido)
3. Estado (índice invertido)
Seleccione el tipo de búsqueda: 1
Ingrese el ID: 02

Valor buscado: ID = 02
Posición obtenida del índice: 1

-----
ID:          02
Nombre:      Pepe
Categoría:   alumno
Descripción: sistemas
Fecha:       2005-11-10
Estado:      activo
-----
Resultado: ENCONTRADO
```

```
-----
Seleccione una opción: 7

=== ÍNDICE PRINCIPAL (ID -> posición) ===
ID: 01 -> Posición: 0
ID: 02 -> Posición: 1
```

## Etapa 11 – Índice invertido

1. **Qué se hace:** consultar registros por categoría con índice invertido.
2. **Comando usado:** menú opción 8 (mostrar) y opción 9 (buscar por categoría).
3. **Qué hace:** generar\_indice\_categoria() crea índices con categoría (id's)  
buscar\_por\_indice\_categoria() devuelve los IDs y sus registros.
4. **Por qué se usa:** buscar por atributo sin recorrer el archivo.
5. **Resultado:** datos clasificados según categoría o estado
6. **Demuestra:** índice invertido sobre el atributo categoría.

```
Seleccione una opción: 8

=== ÍNDICES INVERTIDOS ===

POR CATEGORÍA (categoría -> IDs)
  quiza -> ['01']
  alumno -> ['02', '03']

POR ESTADO (estado -> IDs)
  intermitente -> ['01']
  activo -> ['02']
  inactivo -> ['03']
```

```
Seleccione una opción: 9

=== BÚSQUEDA UTILIZANDO ÍNDICES ===
1. ID (índice principal)
2. Categoría (índice invertido)
3. Estado (índice invertido)
Seleccione el tipo de búsqueda: 2
Ingrese la categoría: alumno

Se encontraron 2 registro(s):
02 - Pepe
03 - Juan
```

```
indices > {} indice_estado.json > ...
1  {
2    "intermitente": [
3      |  "01"
4    ],
5    "activo": [
6      |  "02"
7    ],
8    "inactivo": [
9      |  "03"
10   ]
11 }
```

```
indices > {} indice_categoria.json > ...  
1 {  
2   "quiza": [  
3     "01"  
4   ],  
5   "alumno": [  
6     "02",  
7     "03"  
8   ]  
9 }
```

```
def _buscar_por_indice_invertido(ruta, valor):  
    """  
    Consulta un índice invertido (valor -> [IDs]) y  
    recupera cada registro mediante el índice principal.  
    """  
  
    invertido = cargar_indice(ruta) or {}  
    principal = cargar_indice(ruta_indice_id()) or {}  
  
    ids = []  
  
    for clave, lista in invertido.items():  
        if clave.strip().lower() == valor.strip().lower():  
            ids = lista  
            break  
  
    registros = leer_registros()  
  
    resultados = []  
  
    for id_registro in ids:  
        posicion = principal.get(id_registro)  
  
        if posicion is not None and posicion < len(registros):  
            resultados.append(registros[posicion])  
  
    return resultados
```

## Etapa 12 — Organización multillave

1. **Qué se hace:** localizar el mismo registro por ID, categoría y estado.
2. **Comando usado:** menú opción 9 (por ID, por categoría, por estado).
3. **Qué hace:** busca registros relacionados basados en 3 campos: indice\_id.json (clave: id), indice\_categoria.json (clave: categoría), indice\_estado.json (clave: estado).
4. **Por qué se usa:** requisito de multillave, variedad de opciones de búsqueda.
5. **Resultado:** un mismo registro aparece con las tres búsquedas.
6. **Demuestra:** varias claves de acceso para la misma información.

```
Seleccione una opción: 9

=== BÚSQUEDA UTILIZANDO ÍNDICES ===
1. ID (índice principal)
2. Categoría (índice invertido)
3. Estado (índice invertido)
Seleccione el tipo de búsqueda: 1
Ingrese el ID: 02

Valor buscado: ID = 02
Posición obtenida del índice: 1

-----
ID:          02
Nombre:      Pepe
Categoría:   alumno
Descripción: sistemas
Fecha:       2005-11-10
Estado:      activo
-----
Resultado: ENCONTRADO
```

```
Seleccione una opción: 9

=== BÚSQUEDA UTILIZANDO ÍNDICES ===
1. ID (índice principal)
2. Categoría (índice invertido)
3. Estado (índice invertido)
Seleccione el tipo de búsqueda: 2
Ingrese la categoría: alumno

Se encontraron 2 registro(s):
02 - Pepe
03 - Juan
```

```
Seleccione una opción: 9

=== BÚSQUEDA UTILIZANDO ÍNDICES ===
1. ID (índice principal)
2. Categoría (índice invertido)
3. Estado (índice invertido)
Seleccione el tipo de búsqueda: 3
Ingrese el estado: activo

Se encontraron 1 registro(s):
02 - Pepe
```

## Etapa 13 – Hashing

- **Qué se hace:** calcular posición con función hash y buscar con la tabla.
- **Comando usado:** menú opción 11 (tabla hash), 12 (buscar con hash), 13 (colisiones).
- **Qué hace:**  $\text{funcion\_hash(id)} = \text{ID} \% \text{tamaño\_tabla}$ , manejo de colisiones mediante encadenamiento
- **Por qué se usa:** localización por clave sin recorrer, la lista por posición resuelve colisiones.
- **Resultado:** tabla con posiciones 0–9, Ej: si existen 2 ids,  $\text{id1} = 01$  y  $\text{id2} = 101$ , ocurrirá una colisión en donde:  $\text{id1} \rightarrow \text{id2}$  (encadenamiento)
- **Demuestra:** hashing propio, no solo hash() de Python, con manejo de colisiones.

```
Seleccione una opción: 11

=== TABLA HASH ===
Función hash: ID % 10
Colisiones: encadenamiento (lista de IDs por posición)

Posición 0: VACÍO
Posición 1: 01 -> 101
Posición 2: 02
Posición 3: 03
Posición 4: VACÍO
Posición 5: VACÍO
Posición 6: VACÍO
Posición 7: VACÍO
Posición 8: VACÍO
Posición 9: VACÍO
```

```
Seleccione una opción: 12

=== BÚSQUEDA MEDIANTE HASHING ===
Ingrese el ID: 101

Posición calculada:  $101 \% 10 = 1$ 
IDs almacenados en esa posición: ['01', '101']

-----
ID:      101
Nombre:   antimax
Categoría: absoluto
Descripción: si
Fecha:    2005-12-19
Estado:   real
-----
Resultado: ENCONTRADO
```

```
Seleccione una opción: 13

=== COLISIONES ===
Posición 1: ['01', '101']
```

## Etaapa 14 — Integridad SHA-256

1. **Qué se hace:** registrar hashes, alterar un archivo y detectar el cambio.
2. **Comando usado:** menú opción 14 (registrar), editar data/registros.csv a mano, menú opción 15 (verificar).
3. **Qué hace:** `calcular_sha256()` usa `hashlib.sha256` por bloques, `guardar_hashes()` escribe `indices/hashes.json` y `verificar_integridad()` compara actual con el almacenado
4. **Por qué se usa:** detectar modificaciones fuera del sistema y protege `registros.csv`, `configuracion.json`, `indice_id.json`, `indice_categoria.json`, `indice_estado.json`.
5. **Resultado:** mensaje que confirme una modificación inesperada
6. **Demuestra:** huellas digitales persistentes y detección de alteración.

```
Seleccione una opción: 14

=== REGISTRAR INTEGRIDAD ===

Hashes SHA-256 registrados:

Archivo: data/registros.csv
SHA-256: 3ae6c5d8754ee0fcc393826255f3750659475ab5332fe23f808bee1a359f6701

Archivo: config/configuracion.json
SHA-256: 242652ba5e8010b5001f366039f5ebaa3a1c844c653c96c4a7459f06e528261b

Archivo: indices/indice_id.json
SHA-256: cf1daee523ed81aa2114aa14359f6373b48df379c64f0785150a84c0ab0dd8b1

Archivo: indices/indice_categoria.json
SHA-256: a8047f4a52e5b4900e10b10a759ec225f948253303b85e54913376acae2edde3

Archivo: indices/indice_estado.json
SHA-256: b1d6b0d615db8aaecff8315df055634f1498324dc465bf23dd75f4130f7fbb50
```

adición de id 04

```
data > registros.csv
1 id,nombre,categoria,descripcion,fecha,estado
2 01,Melgar,quiza,no,2077-11-15,intermitente
3 02,Pepe,alumno,sistemas,2005-11-10,activo
4 03,Juan,alumno,amigo,2006-05-16,inactivo
5 101,antimax,absoluto,si,2005-12-19,real
6 04,hacker,amenaza,si,2005-01-08,fantasma
7
```

Seleccione una opción: 15

=== VERIFICACIÓN DE INTEGRIDAD ===

-----

Archivo: data/registros.csv

Estado: ALTERADO

Hash registrado: 3ae6c5d8754ee0fcc393826255f3750659475ab5332fe23f808bee1a359f6701

Hash actual: 9df1995d5dd256171b971b857bc8c22bdb7d112bee89c727fd1134f447d66934

-----

Archivo: config/configuracion.json

Estado: OK

Hash registrado: 242652ba5e8010b5001f366039f5ebaa3a1c844c653c96c4a7459f06e528261b

Hash actual: 242652ba5e8010b5001f366039f5ebaa3a1c844c653c96c4a7459f06e528261b

-----

Archivo: indices/indice\_id.json

Estado: OK

Hash registrado: cf1dae523ed81aa2114aa14359f6373b48df379c64f0785150a84c0ab0dd8b1

Hash actual: cf1dae523ed81aa2114aa14359f6373b48df379c64f0785150a84c0ab0dd8b1

-----

Archivo: indices/indice\_categoria.json

Estado: OK

Hash registrado: a8047f4a52e5b4900e10b10a759ec225f948253303b85e54913376acae2edde3

Hash actual: a8047f4a52e5b4900e10b10a759ec225f948253303b85e54913376acae2edde3

-----

Archivo: indices/indice\_estado.json

Estado: OK

Hash registrado: b1d6b0d615db8aaecff8315df055634f1498324dc465bf23dd75f4130f7fbb50

Hash actual: b1d6b0d615db8aaecff8315df055634f1498324dc465bf23dd75f4130f7fbb50

## Etapa 15 – Logs

1. **Qué se hace:** registrar operaciones y consultarlas.
2. **Comando usado:** realizar operaciones anteriores (1, 3, 5, 6, 10, 21, 22) y luego menú opción 16.
3. **Qué hace:** registrar\_log(operacion, descripcion) agrega en logs/sistema.log la línea fecha | hora | OPERACIÓN | descripción; leer\_logs() la muestra.
4. **Por qué se usa:** trazabilidad de acciones hechas por el programa
5. **Resultado:** reportes de cambios y intentos de movimientos del usuario
6. **Demuestra:** registro automático de todas las operaciones relevantes.

```
Seleccione una opción: 21

=== EXPORTAR INFORMACIÓN A XML ===

Información exportada correctamente.
Archivo generado: exports/registros.xml
```

```
Seleccione una opción: 22

=== GENERAR RESPALDO ===

Respaldo generado correctamente.
Nombre: backup_20261001_205301
Ruta: backups\backup_20261001_205301

Archivos respaldados:
- registros.csv
- configuracion.json
- registros.xml
- hashes.json
```

```

=== LOG DEL SISTEMA ===
2026-10-01 | 19:13:23 | INDEX_AUTO | Índice inexistente o desactualizado; reconstruido automáticamente
2026-10-01 | 19:13:23 | HASH_AUTO | Tabla hash inexistente o desactualizada; regenerada automáticamente
2026-10-01 | 19:13:23 | SYSTEM | Inicio de ejecución del sistema
2026-10-01 | 19:13:58 | CREATE | Registro creado con ID 01
2026-10-01 | 19:13:58 | INDEX_GENERATE | Índices generados (alta del registro 01)
2026-10-01 | 19:13:58 | HASH_TABLE | Tabla hash regenerada (alta del registro 01)
2026-10-01 | 19:13:58 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (alta del registro 01)
2026-10-01 | 19:14:07 | EXPORT_XML | Información exportada a exports/registros.xml
2026-10-01 | 19:14:44 | SYSTEM | Cierre normal del sistema
2026-10-01 | 19:21:30 | SYSTEM | Inicio de ejecución del sistema
2026-10-01 | 19:21:36 | QUERY | Consulta de todos los registros (1)
2026-10-01 | 19:24:47 | UPDATE | Registro actualizado con ID 01
2026-10-01 | 19:24:47 | INDEX_GENERATE | Índices generados (modificación del registro 01)
2026-10-01 | 19:24:47 | HASH_TABLE | Tabla hash regenerada (modificación del registro 01)
2026-10-01 | 19:24:47 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (modificación del registro 01)
2026-10-01 | 19:24:49 | QUERY | Consulta de todos los registros (1)
2026-10-01 | 19:34:47 | QUERY | Consulta de todos los registros (1)
2026-10-01 | 19:35:07 | EXPORT_XML | Información exportada a exports/registros.xml
2026-10-01 | 19:50:29 | SEARCH | Búsqueda secuencial exitosa para ID 01 (1 registros recorridos)
2026-10-01 | 19:51:02 | SEARCH_ATTRIBUTE | Búsqueda por estado = 'intermitente': 1 resultado(s)
2026-10-01 | 19:58:10 | QUERY | Consulta del índice principal
2026-10-01 | 19:58:39 | SEARCH_INDEX | Búsqueda por índice invertido de categoría = 'quiza': 1 resultado(s)
2026-10-01 | 19:58:55 | INDEX_REBUILD | Reconstrucción manual de todos los índices
2026-10-01 | 20:10:17 | CREATE | Registro creado con ID 02
2026-10-01 | 20:10:17 | INDEX_GENERATE | Índices generados (alta del registro 02)
2026-10-01 | 20:10:17 | HASH_TABLE | Tabla hash regenerada (alta del registro 02)
2026-10-01 | 20:10:17 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (alta del registro 02)
2026-10-01 | 20:10:24 | SEARCH | Búsqueda secuencial sin resultado para ID 022 (2 registros recorridos)
2026-10-01 | 20:10:31 | SEARCH | Búsqueda secuencial exitosa para ID 02 (2 registros recorridos)
2026-10-01 | 20:10:44 | SEARCH_INDEX | Búsqueda por índice de ID 02: encontrado
2026-10-01 | 20:11:11 | QUERY | Consulta del índice principal

```

```

1 === LOG DEL SISTEMA ===
2 2026-10-01 | 19:13:23 | INDEX_AUTO | Índice inexistente o desactualizado; reconstruido automáticamente
3 2026-10-01 | 19:13:23 | HASH_AUTO | Tabla hash inexistente o desactualizada; regenerada automáticamente
4 2026-10-01 | 19:13:23 | SYSTEM | Inicio de ejecución del sistema
5 2026-10-01 | 19:13:58 | CREATE | Registro creado con ID 01
6 2026-10-01 | 19:13:58 | INDEX_GENERATE | Índices generados (alta del registro 01)
7 2026-10-01 | 19:13:58 | HASH_TABLE | Tabla hash regenerada (alta del registro 01)
8 2026-10-01 | 19:13:58 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (alta del registro 01)
9 2026-10-01 | 19:14:07 | EXPORT_XML | Información exportada a exports/registros.xml
10 2026-10-01 | 19:14:44 | SYSTEM | Cierre normal del sistema
11 2026-10-01 | 19:21:30 | SYSTEM | Inicio de ejecución del sistema
12 2026-10-01 | 19:21:36 | QUERY | Consulta de todos los registros (1)
13 2026-10-01 | 19:24:47 | UPDATE | Registro actualizado con ID 01
14 2026-10-01 | 19:24:47 | INDEX_GENERATE | Índices generados (modificación del registro 01)
15 2026-10-01 | 19:24:47 | HASH_TABLE | Tabla hash regenerada (modificación del registro 01)
16 2026-10-01 | 19:24:47 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (modificación del registro 01)
17 2026-10-01 | 19:24:49 | QUERY | Consulta de todos los registros (1)
18 2026-10-01 | 19:34:47 | QUERY | Consulta de todos los registros (1)
19 2026-10-01 | 19:35:07 | EXPORT_XML | Información exportada a exports/registros.xml
20 2026-10-01 | 19:50:29 | SEARCH | Búsqueda secuencial exitosa para ID 01 (1 registros recorridos)
21 2026-10-01 | 19:51:02 | SEARCH_ATTRIBUTE | Búsqueda por estado = 'intermitente': 1 resultado(s)
22 2026-10-01 | 19:58:10 | QUERY | Consulta del índice principal
23 2026-10-01 | 19:58:39 | SEARCH_INDEX | Búsqueda por índice invertido de categoría = 'quiza': 1 resultado(s)
24 2026-10-01 | 19:58:55 | INDEX_REBUILD | Reconstrucción manual de todos los índices
25 2026-10-01 | 20:10:17 | CREATE | Registro creado con ID 02
26 2026-10-01 | 20:10:17 | INDEX_GENERATE | Índices generados (alta del registro 02)
27 2026-10-01 | 20:10:17 | HASH_TABLE | Tabla hash regenerada (alta del registro 02)
28 2026-10-01 | 20:10:17 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (alta del registro 02)
29 2026-10-01 | 20:10:24 | SEARCH | Búsqueda secuencial sin resultado para ID 022 (2 registros recorridos)
30 2026-10-01 | 20:10:31 | SEARCH | Búsqueda secuencial exitosa para ID 02 (2 registros recorridos)
31 2026-10-01 | 20:10:44 | SEARCH_INDEX | Búsqueda por índice de ID 02: encontrado
32 2026-10-01 | 20:11:11 | QUERY | Consulta del índice principal
33 2026-10-01 | 20:14:26 | QUERY | Consulta de los índices invertidos
34 2026-10-01 | 20:14:43 | QUERY | Consulta de la tabla hash
35 2026-10-01 | 20:15:39 | CREATE | Registro creado con ID 03
36 2026-10-01 | 20:15:39 | INDEX_GENERATE | Índices generados (alta del registro 03)
37 2026-10-01 | 20:15:39 | HASH_TABLE | Tabla hash regenerada (alta del registro 03)
38 2026-10-01 | 20:15:39 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (alta del registro 03)
39 2026-10-01 | 20:15:51 | QUERY | Consulta de los índices invertidos
40 2026-10-01 | 20:19:25 | QUERY | Consulta de los índices invertidos
41 2026-10-01 | 20:19:32 | SEARCH_INDEX | Búsqueda por índice invertido de categoría = 'alumno': 2 resultado(s)
42 2026-10-01 | 20:26:04 | QUERY | Consulta del índice principal
43 2026-10-01 | 20:37:28 | SEARCH_INDEX | Búsqueda por índice de ID 02: encontrado
44 2026-10-01 | 20:37:42 | SEARCH_INDEX | Búsqueda por índice invertido de categoría = 'alumno': 2 resultado(s)
45 2026-10-01 | 20:38:07 | SEARCH_INDEX | Búsqueda por índice invertido de estado = 'activo': 1 resultado(s)
46 2026-10-01 | 20:45:37 | CREATE | Registro creado con ID 101
47 2026-10-01 | 20:45:37 | INDEX_GENERATE | Índices generados (alta del registro 101)
48 2026-10-01 | 20:45:37 | HASH_TABLE | Tabla hash regenerada (alta del registro 101)
49 2026-10-01 | 20:45:37 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (alta del registro 101)
50 2026-10-01 | 20:45:48 | QUERY | Consulta de la tabla hash
51 2026-10-01 | 20:46:01 | SEARCH_HASH | Búsqueda por hash del ID 101 (posición 1): encontrado
52 2026-10-01 | 20:46:09 | QUERY | Consulta de colisiones de la tabla hash
53 2026-10-01 | 20:48:42 | INTEGRITY_REGISTER | Hashes SHA-256 registrados manualmente
54 2026-10-01 | 20:50:16 | INTEGRITY_CHECK | data/registros.csv -> ALTERADO
55 2026-10-01 | 20:50:16 | INTEGRITY_CHECK | config/configuracion.json -> OK
56 2026-10-01 | 20:50:16 | INTEGRITY_CHECK | indices/indice_id.json -> OK
57 2026-10-01 | 20:50:16 | INTEGRITY_CHECK | indices/indice_categoria.json -> OK
58 2026-10-01 | 20:50:16 | INTEGRITY_CHECK | indices/indice_estado.json -> OK

```

## Etapla 16 — Organización tipo pila

1. **Qué se hace:** historial de operaciones con pila LIFO.
2. **Comando usado:** menú opción 17 (mostrar), 18 (TOP), 19 (POP).
3. **Qué hace:** PilaOperaciones (push, pop, peek, esta\_vacia, tamaño) guarda textos, cada operación hace push; el TOP es la última(lifo).
4. **Por qué se usa:** funcionalidad real: historial de operaciones de la sesión (en memoria).
5. **Resultado:** al hacer POP sale primero la última operación y se registra STACK\_POP en el log.
6. **Demuestra:** comportamiento LIFO.

```
Seleccione una opción: 17

=== PILA DE OPERACIONES ===

TOP
|
▼
[ BACKUP - backup_20261001_205301 ]
[ EXPORT_XML - registros.xml ]
[ CREATE - Registro 101 ]
[ SEARCH_INDEX - Registro 02 ]
[ CREATE - Registro 03 ]
[ SEARCH_INDEX - Registro 02 ]
[ SEARCH - Registro 02 ]
[ CREATE - Registro 02 ]
[ SEARCH - Registro 01 ]
[ EXPORT_XML - registros.xml ]
[ UPDATE - Registro 01 ]

Cantidad de operaciones: 11
```

```
Seleccione una opción: 18

=== TOP DE LA PILA ===

Última operación: BACKUP - backup_20261001_205301
```

```
Seleccione una opción: 19

=== POP DE LA PILA ===

Operación retirada: BACKUP - backup_20261001_205301
```

```

Seleccione una opción: 17

=== PILA DE OPERACIONES ===

TOP
|
▼
[ EXPORT_XML - registros.xml ]
[ CREATE - Registro 101 ]
[ SEARCH_INDEX - Registro 02 ]
[ CREATE - Registro 03 ]
[ SEARCH_INDEX - Registro 02 ]
[ SEARCH - Registro 02 ]
[ CREATE - Registro 02 ]
[ SEARCH - Registro 01 ]
[ EXPORT_XML - registros.xml ]
[ UPDATE - Registro 01 ]

Cantidad de operaciones: 10

```

## Etaapa 17 – Metadatos

1. **Qué se hace:** consultar propiedades de un archivo real del sistema.
2. **Comando usado:** menú opción 20 en registros.csv.
3. **Qué hace:** obtener\_metadatos() usa os.stat y stat.filemode: nombre, ruta absoluta, extensión, tamaño, fecha de creación, fecha de modificación y permisos.
4. **Por qué se usa:** requisito de metadatos sobre archivos reales.
5. **Resultado:** ficha del archivo con los 7 datos.
6. **Demuestra:** uso de os, pathlib/stat sobre archivos del proyecto.

```

Seleccione una opción: 20

=== METADATOS DE ARCHIVOS ===

1. data/registros.csv
2. config/configuracion.json
3. indices/indice_id.json
4. indices/indice_categoria.json
5. indices/indice_estado.json
6. indices/tabla_hash.json
7. indices/hashes.json
8. exports/registros.xml
9. logs/sistema.log

Seleccione un archivo: 1

=== INFORMACIÓN DEL ARCHIVO ===
Nombre: registros.csv
Ruta: C:\Users\Max\manejo_implementacion_archivos_url_s202602-2\data\registros.csv
Extensión: .csv
Tamaño: 258 bytes
Fecha de creación: 2026-10-01 19:24:47
Última modificación: 2026-10-01 20:49:55
Permisos: -rw-rw-rw-

```

## Etapla 18 – Respaldos

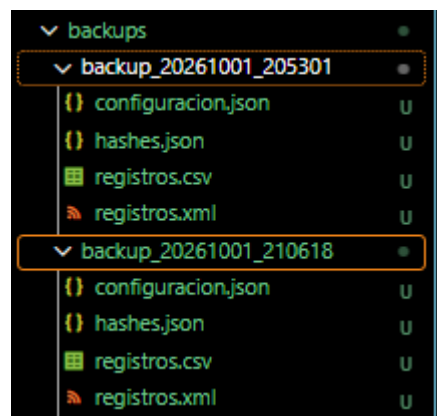
1. **Qué se hace:** generar un respaldo con nombre automático.
2. **Comando usado:** menú opción 22 (generar) y 23 (consultar).
3. **Qué hace:** crear\_respaldo() crea backups/backup\_AAAAMMDD\_HHMMSS/ con registros.csv, configuracion.json, registros.xml y hashes.json (los índices no se copian porque se reconstruyen).
4. **Por qué se usa:** poder volver a un estado anterior.
5. **Resultado:** carpeta backup\_YYYYMMDD\_HHMMSS + log BACKUP.
6. **Demuestra:** respaldo versionado por fecha/hora del sistema.

```
Seleccione una opción: 22

=== GENERAR RESPALDO ===

Respaldo generado correctamente.
Nombre: backup_20261001_210618
Ruta: backups\backup_20261001_210618

Archivos respaldados:
- registros.csv
- configuracion.json
- registros.xml
- hashes.json
```



```
Seleccione una opción: 23

=== RESPALDOS DISPONIBLES ===
1. backup_20261001_210618
2. backup_20261001_205301
```

## Etapa 19 – Restauración

1. **Qué se hace:** modificar datos, restaurar el respaldo y verificar.
2. **Comando usado:** opción 2 (estado antes), opción 6 (eliminar un registro), opción 24 (restaurar, elegir respaldo) , opción 2 y opción 15 en esa secuencia.
3. **Qué hace:** restaurar\_respaldo() valida el respaldo y su CSV, copia los archivos, reconstruye índices y tabla hash y verifica SHA-256.
4. **Por qué se usa:** se demuestra recuperación real.
5. **Resultado:** el registro eliminado reaparece y la tabla hash se reconstruye.
6. **Demuestra:** el flujo obligatorio de respaldo y recuperación.

```
Seleccione una opción: 2

=== REGISTROS ALMACENADOS ===

-----
ID:      01
Nombre:  Melgar
Categoría:  quiza
Descripción:  no
Fecha:    2077-11-15
Estado:   intermitente
-----

-----
ID:      02
Nombre:  Pepe
Categoría:  alumno
Descripción:  sistemas
Fecha:    2005-11-10
Estado:   activo
-----

-----
ID:      03
Nombre:  Juan
Categoría:  alumno
Descripción:  amigo
Fecha:    2006-05-16
Estado:   inactivo
-----

-----
ID:      101
Nombre:  antimax
Categoría:  absoluto
Descripción:  si
Fecha:    2005-12-19
Estado:   real
-----

-----
ID:      04
Nombre:  hacker
Categoría:  amenaza
Descripción:  si
Fecha:    2005-01-08
Estado:   fantasma
-----
```

Seleccione una opción: 6

=== ELIMINAR REGISTRO ===

Ingrese el ID: 05

```
-----  
ID:          05  
Nombre:      Julian  
Categoría:   conserje  
Descripción: chambeador  
Fecha:       1600-10-15  
Estado:      viejo  
-----
```

¿Está seguro de eliminar este registro? (s/n): s

Registro eliminado correctamente.

Índices actualizados correctamente.

Tabla hash actualizada correctamente.

Hashes de integridad actualizados.

Seleccione una opción: 24

=== RESTAURAR RESPALDO ===

Respaldos disponibles:

1. backup\_20261001\_211638
2. backup\_20261001\_210618
3. backup\_20261001\_205301

Seleccione el respaldo: 1

Respaldo seleccionado: backup\_20261001\_211638

¿Desea restaurarlo? (s/n): s

Respaldo restaurado correctamente.

Archivos restaurados:

- data/registros.csv
- config/configuracion.json
- exports/registros.xml
- indices/hashes.json

Índices reconstruidos.

Tabla hash reconstruida.

Verificación de integridad posterior a la restauración:

-----  
Archivo: data/registros.csv

Estado: OK

Hash registrado: c93c6214ec8ade0f6550bdb5100c96d9793a0a36001b08b86c8abeb7826d82cc

Hash actual: c93c6214ec8ade0f6550bdb5100c96d9793a0a36001b08b86c8abeb7826d82cc

-----  
Archivo: config/configuracion.json

Estado: OK

Hash registrado: 242652ba5e8010b5001f366039f5ebaa3a1c844c653c96c4a7459f06e528261b

Hash actual: 242652ba5e8010b5001f366039f5ebaa3a1c844c653c96c4a7459f06e528261b

-----  
Archivo: indices/indice\_id.json

Estado: OK

Hash registrado: 1719bb7c7c651d87be9e023f10b71c9b9914cb5d58f5b11aee75d5967d442fd5

Hash actual: 1719bb7c7c651d87be9e023f10b71c9b9914cb5d58f5b11aee75d5967d442fd5

-----  
Archivo: indices/indice\_categoria.json

Estado: OK

Hash registrado: 782e90a4dd31a7a0e7904ab8f2fdd4716412433cf3a9c72c2881f819d631304b

Hash actual: 782e90a4dd31a7a0e7904ab8f2fdd4716412433cf3a9c72c2881f819d631304b

-----  
Archivo: indices/indice\_estado.json

Estado: OK

Hash registrado: bda6fc16dfcbf3e71d66c4a01c7c9e5c30b951ef77b307e729b053ff75fa6cc1

Hash actual: bda6fc16dfcbf3e71d66c4a01c7c9e5c30b951ef77b307e729b053ff75fa6cc1

-----

-----  
ID: 01  
Nombre: Melgar  
Categoría: quiza  
Descripción: no  
Fecha: 2077-11-15  
Estado: intermitente  
-----

-----  
ID: 02  
Nombre: Pepe  
Categoría: alumno  
Descripción: sistemas  
Fecha: 2005-11-10  
Estado: activo  
-----

-----  
ID: 03  
Nombre: Juan  
Categoría: alumno  
Descripción: amigo  
Fecha: 2006-05-16  
Estado: inactivo  
-----

-----  
ID: 101  
Nombre: antimax  
Categoría: absoluto  
Descripción: si  
Fecha: 2005-12-19  
Estado: real  
-----

-----  
ID: 04  
Nombre: hacker  
Categoría: amenaza  
Descripción: si  
Fecha: 2005-01-08  
Estado: fantasma  
-----

-----  
ID: 05  
Nombre: Julian  
Categoría: conserje  
Descripción: chambeador  
Fecha: 1600-10-15  
Estado: viejo  
-----

Total de registros: 6

## Etapa 20 — Integración del sistema

1. **Qué se hace:** mostrar que todos los módulos trabajan sobre el mismo CSV.
2. **Comando usado:** python main.py → opción 1, luego 7, 8, 11, 15, 16, 22.
3. **Qué hace:** actualizar\_estructuras() tras cada alta/modificación/baja regenera índices, tabla hash y hashes, y escribe el log.
4. **Por qué se usa:** evitar inconsistencias entre archivo principal y estructuras auxiliares.
5. **Resultado:** el registro nuevo aparece en CSV, índice ID, índice categoría/estado, tabla hash, hashes.json y sistema.log.
6. **Demuestra:** unificación de todas las funcionalidades

```
-----
ID:      01
Nombre:  Melgar
Categoría:  quiza
Descripción: no
Fecha:    2077-11-15
Estado:   intermitente
-----

ID:      02
Nombre:  Pepe
Categoría:  alumno
Descripción: sistemas
Fecha:    2005-11-10
Estado:   activo
-----

ID:      03
Nombre:  Juan
Categoría:  alumno
Descripción: amigo
Fecha:    2006-05-16
Estado:   inactivo
-----

ID:      101
Nombre:  antimax
Categoría:  absoluto
Descripción: si
Fecha:    2005-12-19
Estado:   real
-----

ID:      04
Nombre:  hacker
Categoría:  amenaza
Descripción: si
Fecha:    2005-01-08
Estado:   fantasma
-----

ID:      05
Nombre:  Julian
Categoría:  conserje
Descripción: chambeador
Fecha:    1600-10-15
Estado:   viejo
-----

Total de registros: 6
```

exports > registros.xml

```
1  <?xml version='1.0' encoding='utf-8'?>
2  <registros>
3    <registro>
4      <id>01</id>
5      <nombre>Melgar</nombre>
6      <categoria>quiza</categoria>
7      <descripcion>no</descripcion>
8      <fecha>2077-11-15</fecha>
9      <estado>intermitente</estado>
10   </registro>
11   <registro>
12     <id>02</id>
13     <nombre>Pepe</nombre>
14     <categoria>alumno</categoria>
15     <descripcion>sistemas</descripcion>
16     <fecha>2005-11-10</fecha>
17     <estado>activo</estado>
18   </registro>
19   <registro>
20     <id>03</id>
21     <nombre>Juan</nombre>
22     <categoria>alumno</categoria>
23     <descripcion>amigo</descripcion>
24     <fecha>2006-05-16</fecha>
25     <estado>inactivo</estado>
26   </registro>
27   <registro>
28     <id>101</id>
29     <nombre>antimax</nombre>
30     <categoria>absoluto</categoria>
31     <descripcion>si</descripcion>
32     <fecha>2005-12-19</fecha>
33     <estado>real</estado>
34   </registro>
35   <registro>
36     <id>04</id>
37     <nombre>hacker</nombre>
38     <categoria>amenaza</categoria>
39     <descripcion>si</descripcion>
40     <fecha>2005-01-08</fecha>
41     <estado>fantasma</estado>
42   </registro>
43   <registro>
44     <id>05</id>
45     <nombre>Julian</nombre>
46     <categoria>conserje</categoria>
47     <descripcion>chambeador</descripcion>
48     <fecha>1600-10-15</fecha>
49     <estado>viejo</estado>
50   </registro>
51 </registros>
```

```
data > registros.csv
1 id,nombre,categoria,descripcion,fecha,estado
2 01,Melgar,quiza,no,2077-11-15,intermitente
3 02,Pepe,alumno,sistemas,2005-11-10,activo
4 03,Juan,alumno,amigo,2006-05-16,inactivo
5 101,antimax,absoluto,si,2005-12-19,real
6 04,hacker,amenaza,si,2005-01-08,fantasma
7 05,Julian,conserje,chambeador,1600-10-15,viejo
8
```

```
1 {
2   "data/registros.csv": "c93c6214ec8ade0f6550bdb5100c96d9793a0a36001b08b86c8abeb7826d82cc",
3   "config/configuracion.json": "242652ba5e8010b5001f366039f5ebaa3a1c844c653c96c4a7459f06e528261b",
4   "indices/indice_id.json": "1719bb7c7c651d87be9e023f10b71c9b9914cb5d58f5b11aee75d5967d442fd5",
5   "indices/indice_categoria.json": "782e90a4dd31a7a0e7904ab8f2fdd4716412433cf3a9c72c2881f819d631304b",
6   "indices/indice_estado.json": "bda6fc16dfcbf3e71d66c4a01c7c9e5c30b951ef77b307e729b053ff75fa6cc1"
7 }
```

```
indices > {} indice_categoria.json > ...
1 {
2   "quiza": [
3     "01"
4   ],
5   "alumno": [
6     "02",
7     "03"
8   ],
9   "absoluto": [
10    "101"
11  ],
12  "amenaza": [
13    "04"
14  ],
15  "conserje": [
16    "05"
17  ]
18 }
```

```
indices > {} indice_estado.json > ...
1 {
2   "intermitente": [
3     "01"
4   ],
5   "activo": [
6     "02"
7   ],
8   "inactivo": [
9     "03"
10  ],
11  "real": [
12    "101"
13  ],
14  "fantasma": [
15    "04"
16  ],
17  "viejo": [
18    "05"
19  ]
20 }
```

```
indices > {} indice_id.json > ...
1 {
2   "01": 0,
3   "02": 1,
4   "03": 2,
5   "101": 3,
6   "04": 4,
7   "05": 5
8 }
```

```
indices > {} tabla_hash.json > ...  
1  {  
2    "0": [],  
3    "1": [  
4      "01",  
5      "101"  
6    ],  
7    "2": [  
8      "02"  
9    ],  
10   "3": [  
11     "03"  
12   ],  
13   "4": [  
14     "04"  
15   ],  
16   "5": [  
17     "05"  
18   ],  
19   "6": [],  
20   "7": [],  
21   "8": [],  
22   "9": []  
23 }
```

```

=== LOG DEL SISTEMA ===
2026-10-01 | 19:13:23 | INDEX_AUTO | Índice inexistente o desactualizado; reconstruido automáticamente
2026-10-01 | 19:13:23 | HASH_AUTO | Tabla hash inexistente o desactualizada; regenerada automáticamente
2026-10-01 | 19:13:23 | SYSTEM | Inicio de ejecución del sistema
2026-10-01 | 19:13:58 | CREATE | Registro creado con ID 01
2026-10-01 | 19:13:58 | INDEX_GENERATE | Índices generados (alta del registro 01)
2026-10-01 | 19:13:58 | HASH_TABLE | Tabla hash regenerada (alta del registro 01)
2026-10-01 | 19:13:58 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (alta del registro 01)
2026-10-01 | 19:14:07 | EXPORT_XML | Información exportada a exports/registros.xml
2026-10-01 | 19:14:44 | SYSTEM | Cierre normal del sistema
2026-10-01 | 19:21:30 | SYSTEM | Inicio de ejecución del sistema
2026-10-01 | 19:21:36 | QUERY | Consulta de todos los registros (1)
2026-10-01 | 19:24:47 | UPDATE | Registro actualizado con ID 01
2026-10-01 | 19:24:47 | INDEX_GENERATE | Índices generados (modificación del registro 01)
2026-10-01 | 19:24:47 | HASH_TABLE | Tabla hash regenerada (modificación del registro 01)
2026-10-01 | 19:24:47 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (modificación del registro 01)
2026-10-01 | 19:24:49 | QUERY | Consulta de todos los registros (1)
2026-10-01 | 19:34:47 | QUERY | Consulta de todos los registros (1)
2026-10-01 | 19:35:07 | EXPORT_XML | Información exportada a exports/registros.xml
2026-10-01 | 19:50:29 | SEARCH | Búsqueda secuencial exitosa para ID 01 (1 registros recorridos)
2026-10-01 | 19:51:02 | SEARCH_ATTRIBUTE | Búsqueda por estado = 'intermitente': 1 resultado(s)
2026-10-01 | 19:58:10 | QUERY | Consulta del índice principal
2026-10-01 | 19:58:39 | SEARCH_INDEX | Búsqueda por índice invertido de categoría = 'quiza': 1 resultado(s)
2026-10-01 | 19:58:55 | INDEX_REBUILD | Reconstrucción manual de todos los índices
2026-10-01 | 20:10:17 | CREATE | Registro creado con ID 02
2026-10-01 | 20:10:17 | INDEX_GENERATE | Índices generados (alta del registro 02)
2026-10-01 | 20:10:17 | HASH_TABLE | Tabla hash regenerada (alta del registro 02)
2026-10-01 | 20:10:17 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (alta del registro 02)
2026-10-01 | 20:10:24 | SEARCH | Búsqueda secuencial sin resultado para ID 022 (2 registros recorridos)
2026-10-01 | 20:10:31 | SEARCH | Búsqueda secuencial exitosa para ID 02 (2 registros recorridos)
2026-10-01 | 20:10:44 | SEARCH_INDEX | Búsqueda por índice de ID 02: encontrado
2026-10-01 | 20:11:11 | QUERY | Consulta del índice principal
2026-10-01 | 20:14:24 | QUERY | Consulta de los índices invertidos
2026-10-01 | 20:14:43 | QUERY | Consulta de la tabla hash
2026-10-01 | 20:15:39 | CREATE | Registro creado con ID 03
2026-10-01 | 20:15:39 | INDEX_GENERATE | Índices generados (alta del registro 03)
2026-10-01 | 20:15:39 | HASH_TABLE | Tabla hash regenerada (alta del registro 03)
2026-10-01 | 20:15:39 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (alta del registro 03)
2026-10-01 | 20:15:51 | QUERY | Consulta de los índices invertidos
2026-10-01 | 20:19:25 | QUERY | Consulta de los índices invertidos
2026-10-01 | 20:19:32 | SEARCH_INDEX | Búsqueda por índice invertido de categoría = 'alumno': 2 resultado(s)
2026-10-01 | 20:26:04 | QUERY | Consulta del índice principal
2026-10-01 | 20:37:28 | SEARCH_INDEX | Búsqueda por índice de ID 02: encontrado
2026-10-01 | 20:37:42 | SEARCH_INDEX | Búsqueda por índice invertido de categoría = 'alumno': 2 resultado(s)
2026-10-01 | 20:38:07 | SEARCH_INDEX | Búsqueda por índice invertido de estado = 'activo': 1 resultado(s)
2026-10-01 | 20:45:37 | CREATE | Registro creado con ID 101
2026-10-01 | 20:45:37 | INDEX_GENERATE | Índices generados (alta del registro 101)
2026-10-01 | 20:45:37 | HASH_TABLE | Tabla hash regenerada (alta del registro 101)
2026-10-01 | 20:45:37 | INTEGRITY_REGISTER | Hashes SHA-256 actualizados (alta del registro 101)
2026-10-01 | 20:45:48 | QUERY | Consulta de la tabla hash
2026-10-01 | 20:46:01 | SEARCH_HASH | Búsqueda por hash del ID 101 (posición 1): encontrado
2026-10-01 | 20:46:09 | QUERY | Consulta de colisiones de la tabla hash
2026-10-01 | 20:48:42 | INTEGRITY_REGISTER | Hashes SHA-256 registrados manualmente
2026-10-01 | 20:50:16 | INTEGRITY_CHECK | data/registros.csv -> ALTERADO

```

## Etapa 21 — Git durante el desarrollo

1. **Qué se hace:** mostrar el historial con commits significativos.
2. **Comando usado:** git status, git add <archivos>, git commit -m "", git log --oneline --graph
3. **Qué hace:** registra el avance por etapas
4. **Por qué se usa:** el PDF prohíbe un único commit o todos al final.
5. **Resultado:** log de commits
6. **Demuestra:** evolución del proyecto y finalización de funcionamiento.

```
$ git log --oneline
6671664 (HEAD -> 1236925_MELGAR_CASTILLO_MAX_ANDRES) preguntas ReadMe
4525638 funcionamiento completo
14f247c Iniziaci3n de estructura proyecto
9992c0e Merge branch 'devel' of https://github.com/eomarroquinzdocente/manejo_implementacion_archivos_url_s202602 into
1236925_MELGAR_CASTILLO_MAX_ANDRES
05ea2a6 (remote/devel) Documentaci3n inicial del proyecto
5ed1d0e Merge branch 'devel'
fe742c3 Documentaci3n inicial del proyecto
```

- **C3digo a incluir:** commits reales por m3dulo (git add src/archivos.py main.py → commit, git add src/busquedas.py → commit, etc.), captura de git log --oneline y explicaci3n de 3 commits clave.

## Etapa 22 — Ramas

1. **Qué se hace:** desarrollar una funcionalidad en una rama e integrarla.
2. **Comando usado:** git branch, git checkout usuario
3. **Qué hace:** rama de usuario con historial de commits (etapa 21)
4. **Por qué se usa:** aislar una funcionalidad sin afectar main.
5. **Resultado:** ramas main sentado.
6. **Demuestra:** creaci3n, cambio, desarrollo, commit e integraci3n de ramas.

```
vicme@LAPTOP-362BMOJA MINGW64 ~/OneDrive/Escritorio/ManejoEImplementacion/manejo_implementacion_archivos_url_s202602 (
1236925_MELGAR_CASTILLO_MAX_ANDRES)
```

## **Etapas 23 – GitHub**

1. **Qué se hace:** publicar el repositorio remoto.
2. **Comando usado:** git push origin usuario
3. **Qué hace:** enlaza el remoto, sube ramas y commits, sincroniza.
4. **Por qué se usa:** para enviar los documentos finales al repositorio original
5. **Resultado:** historial, ramas y archivos visibles en GitHub.
6. **Demuestra:** repositorio remoto funcional

```
$ git push origin 1236925_MELGAR_CASTILLO_MAX_ANDRES
```

Debido a que esta etapa pide una imagen del push, pero este mismo documento debe estar en la carpeta para el push en sí, no se puede mostrar los resultados del comando mostrado en este documento adjunto a git.

## Etapa 24 – Ejecución final

1. **Qué se hace:** ejecutar el sistema completo desde cero.
2. **Comando usado:** python main.py
3. **Qué hace:** iniciar\_sistema(), usado para inicial el menú que conecta con todo
4. **Por qué se usa:** comprobar que arranca con solo seguir las instrucciones del README.
5. **Resultado:** menú completo visible y opción inválida controlada sin cerrar el programa.
6. **Demuestra:** que el sistema inicia correctamente y presenta todas las opciones requeridas.

```
=====
SISTEMA INTEGRAL DE GESTIÓN Y RECUPERACIÓN DE INFORMACIÓN
Versión 1.0
=====
1. Registrar información
2. Mostrar todos los registros
3. Buscar por ID
4. Buscar por atributo
5. Actualizar registro
6. Eliminar registro
7. Mostrar índice principal
8. Mostrar índice invertido
9. Buscar utilizando índice
10. Reconstruir índices
11. Mostrar tabla hash
12. Buscar utilizando hashing
13. Mostrar colisiones
14. Registrar hashes de integridad
15. Verificar integridad de archivos
16. Consultar logs
17. Mostrar pila de operaciones
18. Consultar TOP de la pila
19. Retirar operación de la pila (POP)
20. Consultar metadatos
21. Exportar información a XML
22. Generar respaldo
23. Consultar respaldos
24. Restaurar respaldo
0. Salir
=====
Seleccione una opción: [ ]
```

los funcionamientos del menú fueron abarcados en las anteriores etapas

**1. ¿Cuál es el archivo principal utilizado para persistir la información?**

Se utiliza registros.csv, en él se guardan los datos inicialmente

**2. ¿Qué diferencia existe entre el archivo CSV y el archivo XML generado?**

CSV es la forma de almacenamiento principal, el XML es una transformación de los datos .csv, a nivel práctico, se diferencian en estructura (XML se rige en etiquetas, CSV tiene un estándar más tabular)

**3. ¿Cómo funciona la búsqueda secuencial?**

Recorre fila por fila hasta encontrar el ID o terminar.

**4. ¿Qué ventaja ofrece un índice frente a recorrer todos los registros?**

Lee solo la fila necesaria, no todo el archivo, lo que ahorra recursos y mejora la eficiencia.

**5. ¿Qué información contiene el índice invertido?**

Relaciona cada valor de atributo con su lista de IDs.

**6. ¿Cómo se implementa la organización multillave?**

Con varios índices en paralelo que se intersectan al buscar.

**7. ¿Qué función hash utiliza el sistema?**

id módulo 10 (tamaño de tabla)

**8. ¿Por qué pueden producirse colisiones?**

Claves distintas pueden dar la misma posición (ej: 01 y 101 módulo 10 dan la misma posición, o sea 1)

**9. ¿Cómo se resuelven las colisiones?**

Encadenamiento, cada posición guarda una lista enlazada.

**10. ¿Qué propósito tiene SHA-256 dentro del sistema?**

Se usa para crear la huella usada para detectar si un archivo fue alterado.

**11. ¿Qué diferencia existe entre el log y la pila de operaciones?**

Log es historial completo; pila guarda operaciones LIFO.

**12. ¿Por qué la pila desaparece cuando termina la ejecución?**

Desaparece ya que la pila existe en la ram

**13. ¿Qué metadatos pueden obtenerse de un archivo?**

Se pueden obtener datos como nombre, ruta, tamaño, extensión, fechas y permisos.

**14. ¿Qué archivos deben considerarse información principal y cuáles son estructuras derivadas?**

El archivo principal es CSV, los derivados son los índices, hashes, logs, pila, XML.

**15. ¿Por qué se reconstruyen los índices después de restaurar un respaldo?**

Porque las posiciones cambian y deben quedar coherentes.